Basic Introduction to Data Structures

◆ What is a Data Structure?

A **Data Structure** is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently.

☞ It helps in:

- Efficient data storage
- Fast searching and sorting
- Better memory usage
- Organized program design

◆ Types of Data Structures

Data Structures are mainly divided into:

1. Linear Data Structures

Data elements are arranged **sequentially (one after another)**.

Examples:

- Array
- Stack
- Queue
- Linked List

2. Non-Linear Data Structures

Data elements are arranged **hierarchically or interconnected**.

Examples:

- Tree
- Graph

Linear Data Structures

1. Array

◆ Definition:

An **Array** is a collection of elements of the same data type stored in **contiguous memory locations**.

◆ Characteristics:

- Fixed size
- Indexed (starts from 0)
- Fast access using index

◆ Example:

```
int arr[5] = {10, 20, 30, 40, 50};
```

Memory Representation:

```
Index:  0   1   2   3   4
Value: 10  20  30  40  50
```

◆ Real-life Example:

Marks of 5 students stored in order.

2. Stack

◆ Definition:

A **Stack** is a linear data structure that follows **LIFO (Last In First Out)** principle.

◆ Operations:

- Push (insert)
- Pop (remove)
- Peek (top element)

◆ Example:

Stack of plates 🍽
Last plate kept is the first removed.

```
Push: 10, 20, 30

Top
30
20
10
```

Pop → 30 removed first.

3. Queue

◆ Definition:

A **Queue** is a linear data structure that follows **FIFO (First In First Out)** principle.

◆ Operations:

- Enqueue (insert)
- Dequeue (remove)

◆ Example:

People standing in a line.

```
Front → 10 20 30 ← Rear
```

Dequeue → 10 removed first.

4. Linked List

◆ Definition:

A **Linked List** is a linear data structure where elements (nodes) are connected using pointers.

Each node contains:

- Data
- Address of next node

◆ Types:

- Singly Linked List
- Doubly Linked List
- Circular Linked List

◆ Example Structure:

```
struct Node {
    int data;
    struct Node *next;
};
```

Representation:

```
10 → 20 → 30 → NULL
```

◆ Advantage:

- Dynamic size
- Easy insertion & deletion

Non-Linear Data Structures

5. Tree

◆ Definition:

A **Tree** is a hierarchical data structure consisting of nodes connected by edges.

◆ Important Terms:

- Root
- Parent
- Child
- Leaf
- Subtree

◆ Example:

```
      10
     /  \
    20   30
   /
  40
```

◆ Real-life Example:

- Folder structure in computer
- Family tree

◆ Special Types:

- Binary Tree
- Binary Search Tree
- AVL Tree

6. Graph

◆ Definition:

A **Graph** is a non-linear data structure consisting of:

- Vertices (Nodes)
- Edges (Connections)

◆ Types:

- Directed Graph
- Undirected Graph
- Weighted Graph

◆ Example:

```
A ── B
|     |
C ── D
```

◆ Real-life Example:

- Social networks
- Google Maps routes
- Flight networks